

Checklist prácticas de construcción

HTML		
Criterio	Estado	Justificación
Estructura HTML5 semántica	Implementado	Uso de etiquetas semánticas como <main>, <section>, <article>, <header>, <footer> para organizar el contenido de las vistas.
Separación de responsabilidades	Implementado	El contenido HTML, los estilos CSS y la lógica JavaScript se encuentran en archivos independientes.
Formularios HTML válidos	Implementado	Uso de etiquetas <form>, <input>, <label>, <button> y atributos de validación para captura de datos del usuario.
Acceso al DOM mediante identificadores (id)	Implementado	Los elementos HTML poseen atributos id y class para su manipulación mediante JavaScript.
Organización modular de vistas	Implementado	Cada funcionalidad posee su propia vista HTML independiente (perfil, alimentación, rutinas, calculadora, acceso, etc.).
Pruebas de calidad del código	Parcialmente exitoso	Al realizar las pruebas de calidad empleando lint se detectaron 47 errores totales distribuidos en todos algunos documentos html.
CSS		
Criterio CSS	Estado	Evidencia
Metodología BEM	Implementado	Uso de nomenclatura basada en bloques, elementos y modificadores como

		tarjetaEntrenamiento_ _actividad, barraProgreso__rellen o y tarjetaObjetivos__val or.
Arquitectura ITCSS	Implementado	Organización de estilos en archivos separados para configuraciones globales, componentes y vistas específicas.
Variables CSS reutilizables	Implementado	Definición de propiedades personalizadas dentro de :root para colores, tipografías y estilos globales reutilizables.
Reutilización de estilos	Implementado	Creación de clases reutilizables aplicadas en múltiples componentes para evitar duplicación de código.
Herencia de estilos	Implementado	Aprovechamiento de propiedades heredables como tipografía, color y alineación para reducir reglas redundantes.
Separación de estilos por responsabilidad	Implementado	Archivos CSS independientes para páginas principales y componentes modales.
Diseño responsivo mediante Grid Layout	Implementado	Uso de CSS Grid para organizar la distribución de componentes en distintas vistas del sistema.
Uso de Grid Areas	Implementado	Definición de áreas específicas mediante grid-template-areas para estructurar visualmente los paneles y tarjetas.
Uso de Flexbox	Implementado	Implementación de Flexbox para alineación, distribución y posicionamiento de elementos internos.
Consistencia visual	Implementado	Uso centralizado de colores, tamaños y tipografías mediante variables globales.

Modularidad de componentes	Implementado	Cada tarjeta, formulario y modal posee estilos propios sin afectar otros componentes.
Reutilización de clases	Implementado	Los cambios de color, tipografía y espaciado pueden realizarse desde variables globales sin modificar múltiples archivos.
Pruebas de calidad del código css	Parcialmente exitosas	La herramienta empleada, Stylelint, detectó un total de 948 potenciales errores relacionados con convenciones de nomenclatura, comentarios y formato del código.

JavaScript Frontend

Criterio	Estado	Evidencia
ECMAScript moderno	Implementado	Uso de class, import/export, async/await, funciones flecha y módulos ES6.
Programación Orientada a Objetos	Implementado	Implementación de clases Controlador, Servicio y Modelo.
Separación de responsabilidades	Implementado	La lógica se distribuye entre modelos, servicios y controladores.
Manipulación del DOM	Implementado	Uso de getElementById, querySelector y eventos para interactuar con la interfaz.
Programación asíncrona	Implementado	Consumo de APIs mediante fetch y async/await.
Validación de formularios	Implementado	Verificación de datos antes de enviarlos al backend.
Persistencia de sesión	Implementado	Uso de localStorage para almacenar JWT e información del usuario.
Pruebas unitarias	Implementado	Uso de Vitest para validar lógica de negocio y

		modelos.
Pruebas unitarias de código	Implementado	Análisis de los models mediante ESLint
Backend		
Criterio	Estado	Evidencia
Arquitectura REST	Implementado	Uso de endpoints organizados por recursos.
Métodos HTTP	Implementado	Implementación de GET, POST, PUT y DELETE.
Intercambio de datos JSON	Implementado	Comunicación cliente-servidor mediante JSON.
Node.js y Express	Implementado	Desarrollo de API utilizando Express.
ES Modules	Implementado	Uso de <code>import</code> y <code>export</code> en el proyecto.
Arquitectura por capas	Implementado	Separación en rutas, controladores, servicios y modelos.
Middleware	Implementado	Uso de middlewares para autenticación y procesamiento de solicitudes.
Autenticación JWT	Implementado	Protección de recursos mediante tokens JWT.
Encriptación de contraseñas	Implementado	Uso de <code>bcrypt</code> para almacenamiento seguro de contraseñas.
Validación de datos	Implementado	Verificación de información recibida antes de procesarla.
Convención camelCase	Implementado	Variables, funciones y métodos siguen nomenclatura camelCase.
Calidad de código	Implementado	Se configuró y ejecutó ESLint para análisis estático del código. De igual forma, se generaron reportes de errores.